

ICS-PA4-1 实验报告

241220029 杨智宇

思考题

1. 详细描述从测试用例中的 `int $0x80` 开始一直到 `HIT_GOOD_TRAP` 为止的详细系统行为（完整描述控制的转移过程，即相关函数的调用和关键参数传递过程），可以通过文字或画图的方式来完成；

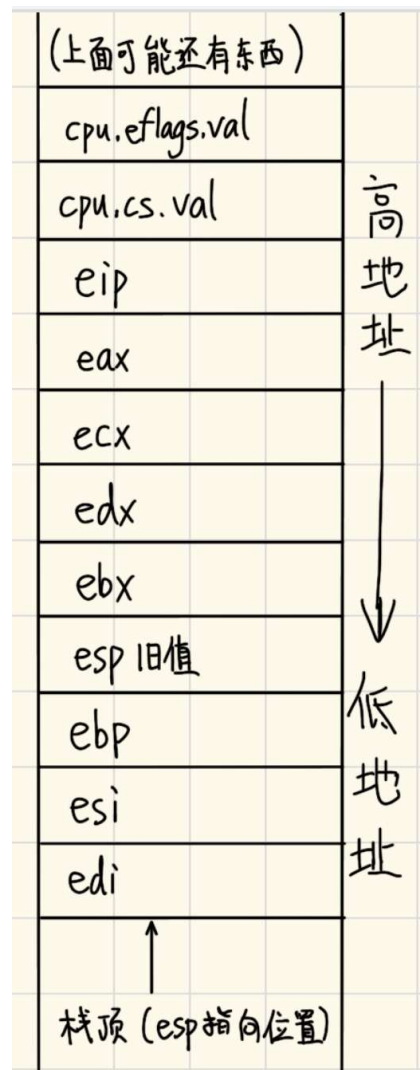
答：当测试用例执行到 `int $0x80` 指令时，`int` 指令通过 `raise_sw_intr()` 调用 `raise_intr()` 函数，从而 CPU 会查找中断向量表（IDT），并根据中断号 `0x80` 找到对应的门描述符，将当前的 EFLAGS 寄存器、代码段选择子（CS）和指令指针（EIP）压入栈（由 `esp` 表示栈指针来实现），保存当前的上下文，以及更新 CS 和 EIP 以指向中断服务例程的起始位置，这样就进入了内核态并开始执行内核代码。

在内核态下，执行 `do_irq.S` 中的代码（在此阶段前，会先经过一个 `<vec>` 的阶段，每一个 `<vec>` 由三行汇编代码构成，连续 `push` 两次后跳转至 `asm_do_irq`），这部分代码首先会通过 `pusha` 指令保存所有通用寄存器的状态，`push esp` 用于保存当前栈指针，接着调用 `irq_handle` 来处理特定的中断或异常。调用 `do_syscall` 函数进行系统调用（`tf` 又当作参数传给了 `do_syscall`），输出“Hello,world!”。

完成所有这些处理后，程序需要恢复之前保存的现场信息，即从栈中弹出先前保存的寄存器值。先执行与 `pusha` 指令对应的 `popa` 指令恢复通用寄存器，再执行一条 `iret` 指令（中断返回）`pop` 出 EIP、CS 和 EFLAGS，使得控制权返回到用户空间，在继续执行 `int $0x80` 之后的下一条指令。在 `hello-inline` 测试样例中即为 `HIT_GOOD_TRAP`。

2. 在描述过程中，回答 `kernel/src/irq/do_irq.S` 中的 `push %esp` 起什么作用，画出在 `call irq_handle` 之前，系统栈的内容和 `esp` 的位置，指出 `TrapFrame` 对应系统栈的哪一段内容。

答：`push esp` 即用于保存当前栈指针，其指向的 `TrapFrame` 结构包含了系统调用前的处理器状态。在 `call irq_handle` 之前一个可能的系统栈示意图如右图所示：



3. 详细描述 NEMU 和 Kernel 响应时钟中断的过程和先前的系统调用过程不同之处在哪里？相同的地方又在哪里？可以通过文字或画图的方式来完成。

答：（一）NEMU 和 Kernel 响应时钟中断的过程和先前的系统调用过程不同之处有：从触发方式看，系统调用由用户程序通过特定指令（如 `int $0x80`）主动发起请求操作系统服务；时钟中断由硬件定时器自动产生通知 CPU 时间已经流逝了一定的数量。从中断号看，系统调用中断号通常是固定的（如 0x80），并且是通过软件指令明确指定的；时钟中断的中断号由硬件决定，对于标准 PC 架构时钟中断一般对应于 IRQ 0，会映射到 IDT 中的某个特定位置（ $32 + \text{IRQ}$ ）。从处理逻辑看，系统调用内核根据传入的系统调用号执行相应的服务例程，之后返回到用户空间继续执行；时钟中断则可能处理与时钟相关的任务。

（二）NEMU 和 Kernel 响应时钟中断的过程和先前的系统调用过程相同之处有：中断处理流程无论是系统调用还是时钟中断，都会经历类似的步骤来保存现场信息：如通过 `raise_intr()` 函数来 push EFLAGS、CS、EIP，查找并跳转至相应的中断处理程序，处理完毕后恢复现场信息并通过 `iret` 指令返回；两者都会导致 CPU 从用户态进入内核态，以允许内核代码执行关键操作。